



NORTH SOUTH UNIVERSITY
DEPARTMENT OF ELECTRICAL AND COMPUTER ENGINEERING

PROJECT REPORT OF

Personal Expense Tracker

MY SPHERE

Java Swing Project for *CSE215: Programming Language II, Section: 10*

PROJECT SUBMITTED BY

Rejoan Hasan Mugdho
ID: 253 1257042

Md Parvez Mosharraf Bhuiyan
ID: 253 1245642

Tanzim Mohammad Enayetullah
ID: 253 1761642

PROJECT SUBMITTED TO

Dr. Shamim Al Mamun [SAM3]

Professor, North South University
Department of Electrical and Computer Engineering

CSE215.10

Table of Contents

1. Introduction
2. Project Overview
3. Problem Statement
4. Objectives
5. Proposed Solution
6. Key Features
7. Technology Stack
8. System Architecture
9. Object-Oriented Programming Concepts
10. System Modules
11. Database and File Management
12. User Interface Design
13. Application Screenshots
14. System Workflow
15. Input Validation and Exception Handling
16. Testing and Results
17. Conclusion
18. References

1. Introduction

Managing personal finances effectively is an important part of everyday life. Students and individuals frequently need to keep track of their income, expenses, bills, purchases, and savings. Maintaining such records manually can become inconvenient and may lead to inaccurate calculations or loss of important information.

To address this problem, **MySphere – Personal Finance & Expense Tracker** has been developed as a Java-based desktop application. The application provides users with a simple graphical interface through which they can record income and expenses, monitor their balance, review recent transactions, and modify or delete previously recorded transactions.

The project is implemented using **Java Swing** for the graphical user interface and uses **CSV files** for persistent storage of user information and transaction records. The project also demonstrates important Object-Oriented Programming concepts including abstraction, inheritance, polymorphism, encapsulation, interfaces, exception handling, and file handling.

The source project identifies itself as a Java Swing theory project, while the implemented interface uses the application name **MySphere**.

2. Project Overview

MySphere is a personal finance management application designed to provide users with a convenient way to manage financial transactions.

The system allows users to:

- Create an account.
- Log into the application.
- Record income.
- Record expenses.
- View total balance.
- View total income.
- View total spending.
- Review recent transactions.
- Edit existing transactions.
- Delete transactions.
- Store transaction records using CSV files.
- Log out from the application.
- View information about the application and its developers.

The application's main dashboard displays a greeting, the user's name, total balance, income, spending, and recent transactions. The source code implements separate UI panels for authentication, adding transactions, editing transactions, and displaying application information.

3. Problem Statement

Many individuals, particularly students, have difficulty maintaining an organized record of their daily financial activities. Recording transactions manually in notebooks or spreadsheets can be time-consuming and makes automatic calculation of total income, expenses, and balance difficult.

Common problems include:

- Forgetting to record daily expenses.
- Difficulty calculating the current balance.
- Lack of organized transaction history.
- Difficulty correcting previously entered information.
- Difficulty maintaining separate records for different users.
- Lack of a simple desktop-based financial management tool.

Therefore, a lightweight application is required that can provide basic financial tracking functionality through an easy-to-use graphical interface.

4. Objectives

The primary objectives of MySphere are:

1. To develop a user-friendly personal finance management application.
2. To implement the application using Java and Java Swing.
3. To demonstrate practical application of Object-Oriented Programming principles.
4. To allow users to record both income and expenses.
5. To automatically calculate total income, total spending, and balance.
6. To provide persistent storage using CSV files.
7. To implement user authentication and account creation.
8. To allow users to edit and delete existing transactions.
9. To validate user input and handle common errors.
10. To create a clean and understandable graphical user interface.

5. Proposed Solution

The proposed solution is a desktop application developed using Java Swing.

The system separates the application into logical components. The core transaction-related functionality is implemented through classes such as Transaction, Income, Expense, and TransactionManager, while the graphical interface is divided into several UI classes.

The application follows the following general process:

User → Authentication → Dashboard → Transaction Management → CSV Storage

When a user logs in successfully, the application initializes a TransactionManager for that user. The manager loads the user's transaction data from a CSV file and calculates the current income, spending, and balance. New transactions can then be added, edited, or deleted through the graphical interface.

6. Key Features

6.1 User Registration

New users can create accounts by providing:

- Full Name
- Username
- Password

The system checks whether the username already exists and requires the password to contain at least eight characters.

6.2 User Login

Registered users can log into the application using their username and password.

The system distinguishes between:

- Username not found
- Incorrect password
- Successful authentication

After successful authentication, the user is taken to the main application dashboard.

6.3 Income Management

Users can add income records by specifying:

- Date
- Description
- Amount

The system creates an Income object and adds it to the user's transaction list.

6.4 Expense Management

Users can similarly record expenses using the same transaction information.

The system creates an Expense object and calculates its financial impact as a negative amount.

6.5 Balance Calculation

The application automatically calculates:

Balance = Total Income – Total Spending

The TransactionManager recalculates these values by processing every transaction and determining whether its impact is positive or negative.

6.6 Transaction Editing

Existing transactions can be selected from the transaction list and modified.

Users can update:

- Transaction type
- Date
- Description
- Amount

The updated transaction is then saved back into the CSV file.

6.7 Transaction Deletion

Users can delete an existing transaction from the editing interface. The transaction is removed from memory and the CSV file is rewritten accordingly.

6.8 Persistent Storage

The application uses CSV files to store transaction information. Each user has a separate transaction file based on their username.

A transaction is stored in the following general format:

Date, Type, Description, Amount

The generateCSV() method produces this format automatically.

7. Technology Stack

Technology	Purpose
Java	Core programming language
Java Swing	Graphical User Interface
Java AWT	GUI components and event handling
CSV	Persistent data storage
File I/O	Reading and writing application data
Object-Oriented Programming	Application architecture
CardLayout	Switching between UI panels

The project makes extensive use of Java Swing components such as JFrame, JPanel, JButton, JTextField, JPasswordField, JLabel, JScrollPane, and CardLayout.

8. System Architecture

The application can be conceptually divided into three major layers.

8.1 Presentation Layer

This layer contains the graphical user interface.

Major classes include:

- Auth_UI
- App_UI
- AddTransactionPanel
- EditTransactions_UI
- Card_UI
- About_UI
- Style

8.2 Business Logic Layer

This layer manages transaction processing and calculations.

Major classes include:

- Transaction
- Income
- Expense
- TransactionManager

8.3 Data Storage Layer

The application uses CSV files for persistent storage.

Examples include:

- users.csv
- <username>_transactions.csv

The transaction manager loads existing records from CSV files when a user starts the application and rewrites the file after editing or deleting transactions.

9. Object-Oriented Programming Concepts

One of the major purposes of the project is to demonstrate practical implementation of Object-Oriented Programming.

9.1 Abstraction

Transaction is declared as an abstract class.

It defines the common properties and behavior of transactions while leaving calculateImpact() as an abstract method.

Transaction

```
|  
+ ---- Income  
  
|  
+ ---- Expense
```

This provides a common structure for different transaction types.

9.2 Inheritance

Both Income and Expense extend the Transaction class.

Therefore, common transaction properties such as date, description, and amount are inherited from the parent class.

9.3 Polymorphism

The application stores different transaction types using the common Transaction reference:

List<Transaction>

Therefore, an Income object or an Expense object can be treated as a Transaction.

The overridden calculateImpact() method provides different behavior for each subclass.

- Income → positive impact
- Expense → negative impact

This is a practical example of runtime polymorphism.

9.4 Encapsulation

The fields of the Transaction class are declared private:

private String date;

private String description;

private double amount;

Access is provided through getter methods such as:

- getDate()
- getDes()
- getAmount()

This protects the internal state of the object and demonstrates encapsulation.

9.5 Interface

The project defines an Exportable interface containing:

generateCSV()

The abstract Transaction class implements this interface, allowing transaction objects to generate their CSV representation.

10. System Modules

Module 1: Authentication Module

Responsible for:

- Login
- Account creation
- Username verification
- Password verification
- Password-length validation
- Logout

The authentication module stores user information in users.csv.

Module 2: Dashboard Module

The dashboard provides the user's financial overview.

It displays:

- Greeting
- User name
- Total balance
- Total income
- Total spending
- Recent transactions
- Logout option

The dashboard is implemented primarily through App_UI.

Module 3: Transaction Entry Module

The transaction entry module allows users to create new income or expense records.

The interface contains:

- Expense/Income selector
- Date field
- Description field
- Amount field
- Save button
- Cancel button

The date is validated using Java's LocalDate functionality, while the amount is checked to ensure it is numeric.

Module 4: Transaction Management Module

The transaction management module provides editing and deletion functionality.

Users can:

- Open a transaction.
 - Change its type.
 - Modify the date.
 - Modify the description.
 - Modify the amount.
 - Save changes.
 - Delete the transaction.
-

Module 5: Data Management Module

TransactionManager handles:

- Loading transactions
- Adding transactions
- Editing transactions
- Deleting transactions
- Recalculating totals
- Writing transactions to CSV

This class acts as the central manager for a user's financial records.

Module 6: UI Styling Module

The Style class centralizes visual properties such as:

- Fonts
- Colors
- Buttons
- Input field colors

The project uses the **Poppins** font and a green-based visual theme.

11. Database and File Management

Instead of using a relational database, MySphere uses CSV-based file storage.

User Data

The application stores user information in:

users.csv

The general structure is:

Full Name, Username, Password

Transaction Data

Each user receives a separate transaction file:

user_transaction/<username>_transactions.csv

This design provides basic separation of user transaction records.

When the application starts, TransactionManager loads the user's existing transaction file. When a new transaction is added, it is appended to the file. When a transaction is edited or deleted, the file is rewritten using the current transaction list.

12. User Interface Design

The project uses **Java Swing** to create a desktop graphical user interface.

The interface is designed around simplicity and ease of navigation.

The main navigation contains:

- **Home**
- **Add**
- **About**

The application uses CardLayout to switch between different panels without opening multiple application windows.

Visual Design

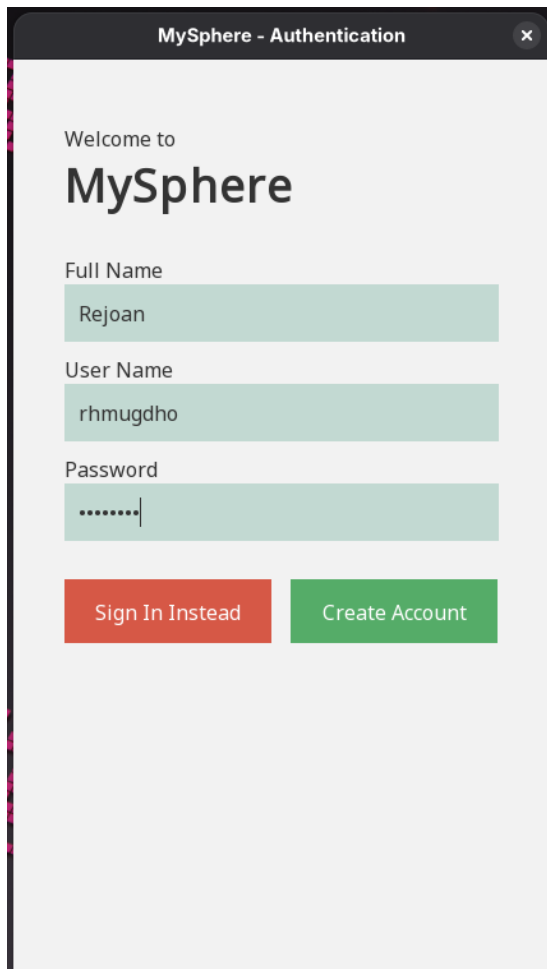
The application uses:

- Green as the primary theme color.
- Off-white backgrounds.
- Green for income-related elements.
- Red for expense-related elements.
- Poppins typography.
- Large balance display.
- Card-style transaction records.

This provides a clean visual distinction between income and expenses.

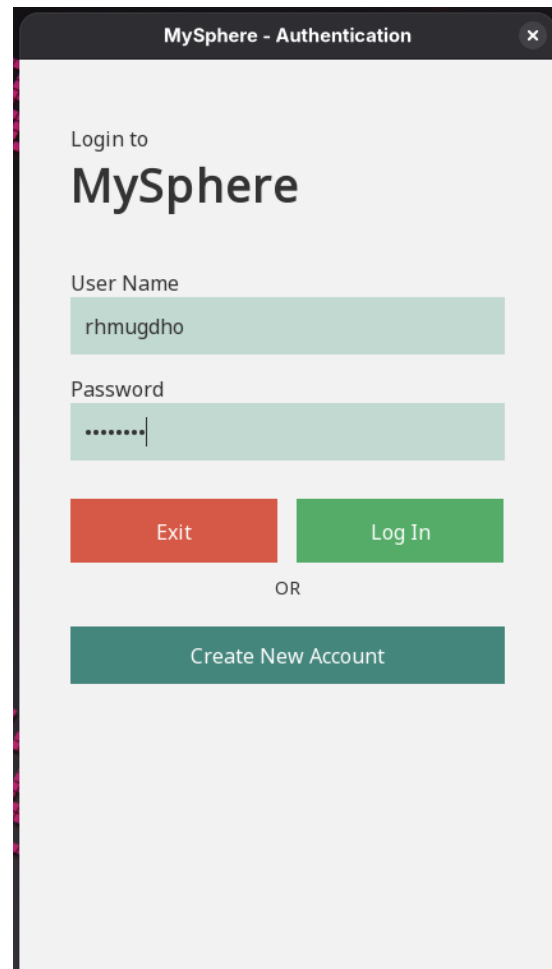
13. Application Screenshots

The following figures present the principal user-interface screens of MySphere. They demonstrate the authentication flow, financial dashboard, transaction entry, transaction editing, and application information interfaces implemented in the Java Swing application.



The screenshot shows a Java Swing window titled "MySphere - Authentication". The window has a light gray background. At the top, it says "Welcome to" followed by "MySphere" in a large, bold font. Below this, there are three text input fields: "Full Name" with the text "Rejoan", "User Name" with the text "rh mugdho", and "Password" with masked characters ".....". At the bottom, there are two buttons: a red button labeled "Sign In Instead" and a green button labeled "Create Account".

Figure 1. User Registration Interface



The screenshot shows a Java Swing window titled "MySphere - Authentication". The window has a light gray background. At the top, it says "Login to" followed by "MySphere" in a large, bold font. Below this, there are three text input fields: "User Name" with the text "rh mugdho", "Password" with masked characters ".....", and a "Log In" button. Below the "Log In" button, there is a red button labeled "Exit". Below the "Exit" button, there is a green button labeled "Log In". Below the "Log In" button, there is a dark green button labeled "Create New Account".

Figure 2. User Login Interface

13. Application Screenshots (Continued)

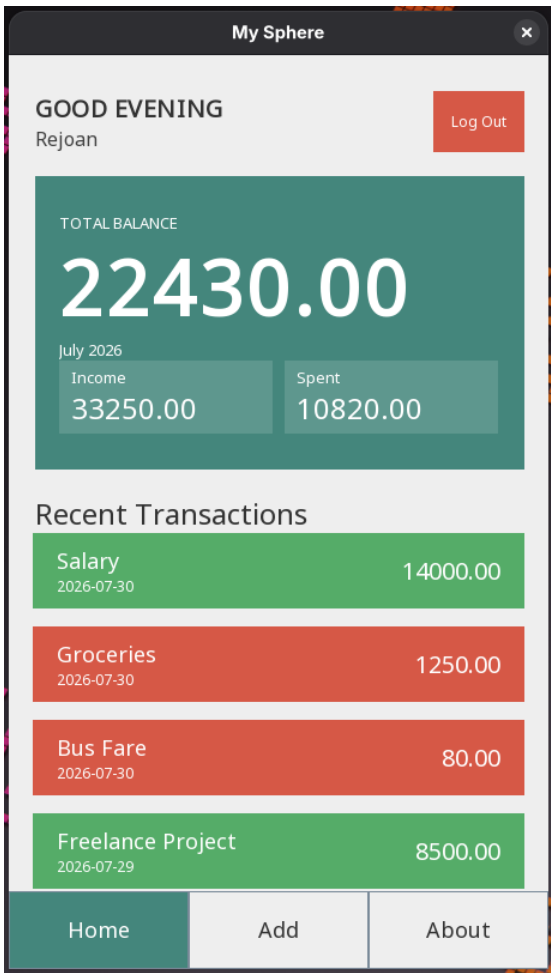


Figure 3. Main Dashboard and Financial Overview

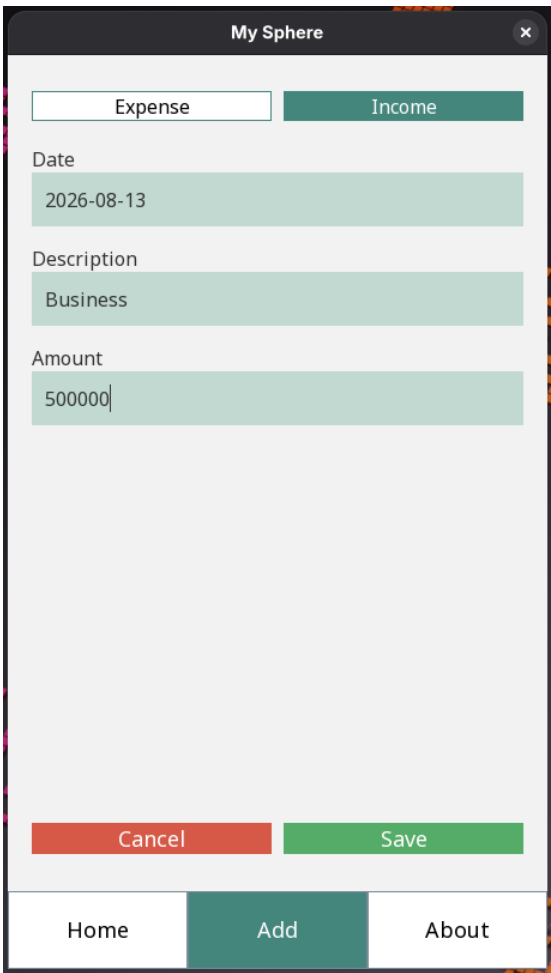


Figure 4. Add Transaction Interface – Expense

13. Application Screenshots (Continued)

The screenshot shows a mobile application window titled "My Sphere" with a close button (X) in the top right corner. The interface is for adding a transaction. At the top, there are two buttons: "Expense" (dark green) and "Income" (white with a green border). Below these are four text input fields: "Date" with the value "2026-08-13", "Description" with the value "Tour", and "Amount" with the value "30000". At the bottom of the form area are two buttons: "Cancel" (red) and "Save" (green). The bottom navigation bar has three items: "Home" (white), "Add" (dark green), and "About" (white).

Figure 5. Add Transaction Interface – Income

The screenshot shows a mobile application window titled "My Sphere" with a close button (X) in the top right corner. The interface is for editing and deleting a transaction. At the top, there is a "Select Type" section with two buttons: "Expense" (white with a green border) and "Income" (dark green). Below this are three text input fields: "Date" with the value "2026-07-30", "Description" with the value "Salary", and "Amount" with the value "14000.00". Below these fields are two buttons: "Cancle" (white with a green border) and "Save" (green). Below these buttons is a red button labeled "Delete Transaction". The bottom navigation bar has three items: "Home" (dark green), "Add" (white), and "About" (white).

Figure 6. Edit and Delete Transaction Interface

13. Application Screenshots (Continued)

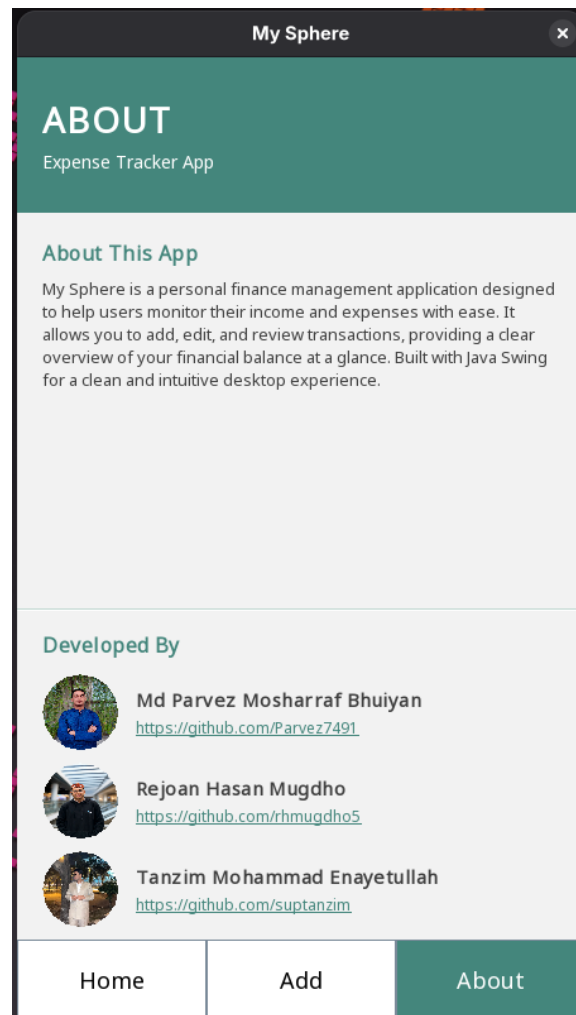
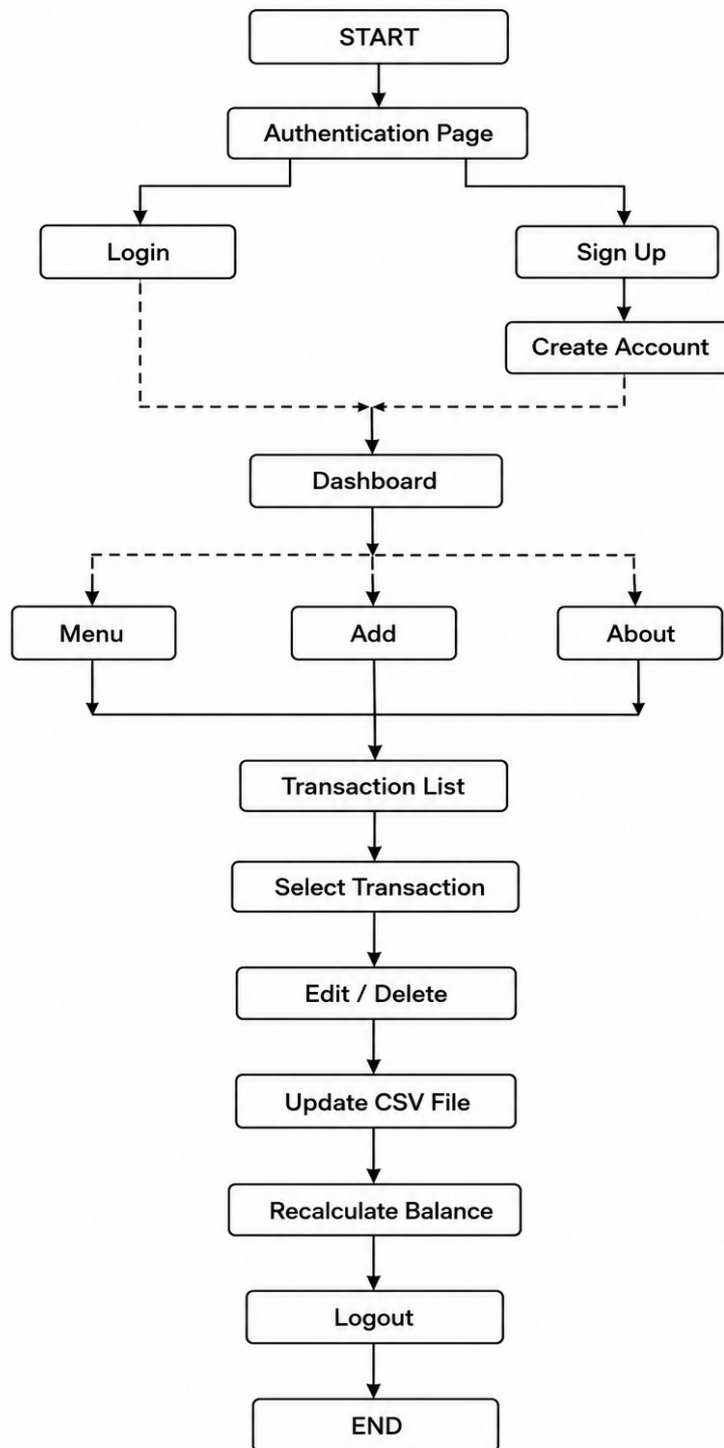


Figure 7. About and Development Team Interface

14. System Workflow

The overall system workflow is:



15. Input Validation and Exception Handling

Input validation is implemented to prevent invalid data from being entered.

Username Validation

The system checks whether:

- Username is empty.
- Username already exists.

Password Validation

During registration, the password must contain at least eight characters.

Amount Validation

The system uses `Double.parseDouble()` to ensure that the entered amount is numeric.

If the input is invalid, an error dialog is displayed.

Date Validation

The application validates dates using:

`LocalDate.parse(date)`

An invalid date results in an error message instructing the user to use the YYYY-MM-DD format.

File Handling Exceptions

The application handles `IOException` during reading and writing operations and also handles `NumberFormatException` while processing numerical transaction data.

16. Testing and Results

The application was designed with several functional scenarios in mind.

Test Case	Expected Result
Empty username during login	Error message displayed
Incorrect password	Password mismatch message
New account creation	Account successfully created
Duplicate username	Error message displayed
Password shorter than 8 characters	Registration rejected
Valid income entry	Income added successfully
Valid expense entry	Expense added successfully
Invalid amount	Input error displayed
Invalid date	Date error displayed
Edit transaction	Transaction updated
Delete transaction	Transaction removed
Logout	User returned to authentication interface
Existing CSV records	Loaded when application starts

The project also contains example transaction records demonstrating both income and expense entries, such as salary, freelance projects, groceries, bills, transportation, and other personal expenses.

17. Conclusion

MySphere – Personal Finance & Expense Tracker is a Java Swing-based desktop application developed to simplify personal financial management.

The project successfully demonstrates the practical application of Java programming and Object-Oriented Programming concepts. Through the use of abstraction, inheritance, polymorphism, encapsulation, interfaces, file handling, and exception handling, the project provides a strong practical demonstration of concepts learned in an academic Java/OOP course.

The application allows users to register and log in, record income and expenses, view their financial balance, review transaction history, edit or delete records, and maintain persistent transaction data through CSV files.

Beyond its functional capabilities, the project also demonstrates the importance of separating user-interface components from core transaction-management logic. With future improvements such as database integration, secure password hashing, financial analytics, budgeting, reporting, and cloud synchronization, MySphere could be developed into a more complete personal finance management platform.

Overall, the project provides a practical and meaningful implementation of Java Swing and Object-Oriented Programming while addressing a real-world problem of personal financial tracking.

18. References

1. Oracle. *Java Documentation – Java Platform, Standard Edition*.
2. Oracle. *Java Swing Documentation*.
3. Oracle. *Java AWT and Event Handling Documentation*.
4. Oracle. *Java I/O Documentation*.
5. Project source code and implementation files developed by the project team.

Appendix A: Major Project Classes

The project contains the following major classes and interfaces:

Core Classes

- Transaction
- Income
- Expense
- TransactionManager
- Exportable

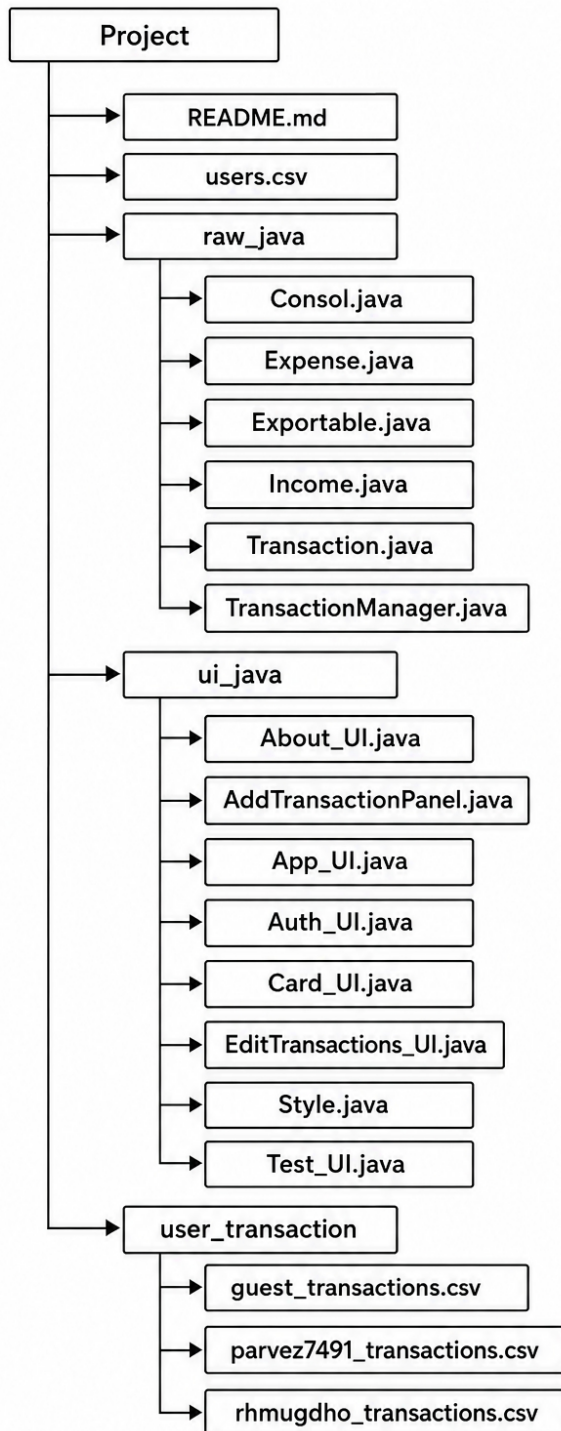
User Interface Classes

- Auth_UI
- App_UI
- AddTransactionPanel
- EditTransactions_UI
- Card_UI
- About_UI
- Style
- Test_UI

These classes collectively implement the application's authentication, transaction processing, persistence, and graphical interface functionality.

Appendix B: Project Structure

Project



This structure separates the core transaction logic from the graphical user-interface components, making the project easier to understand and maintain.